Foundations of R Chapter 8

Robert Gulotty

11/10/2018

This chapter covers the manipulation of strings and how string manipulation can be used to scrape websites. We have seen already that strings are a kind of datatype in R, made up of characters or letters, and it turns out that this is the language of the internet.

Most of the prior chapters involved some basic string manipulaiton. We made strings by using the `as.character()` command, by encasing our data in quotes, or by using the ubiquitous `paste`. Here we describe in more depth the most useful package for string data: `stringr`. It is pronounced the same way as Idriss Elba's character in the Wire.

Stringr commands take as input a string or string vector, a regular expression, and some control commands. The following,`str_replace`, is typical. `str_replace` finds the pattern in the second entry, and replaces it with the third entry.

```
str_replace("banana", "n", "f")
```

```
## [1] "bafana"
```

```
str_replace_all("banana", "n", "f")
```

```
## [1] "bafafa"
```

We can also extract things from a vector of strings

```
str_extract_all("banana", "n")
```

```
## [[1]]
## [1] "n" "n"
```

```
str_extract_all(c("Alabama", "Alaska", "Arizona", "Arkansas"), "a")
```

```
## [[1]]
## [1] "a" "a" "a"
##
## [[2]]
## [1] "a" "a"
##
## [[3]]
## [1] "a"
##
## [[4]]
## [1] "a" "a"
```

Notice it returns a list, which we can turn into a simple vector with the `unlist` command.

So far, we have been looking for patterns in the data that are simple, single letters and the like. However, as you have already seen, there is a powerful programming language for pattern regognition in strings called regular expressions. This computer language is embedded in many other software packages, and is one of the most compact (and most difficult to read) programming languages. You can place regular expressions in many places that do patterns in R.

Regular Expressions

Square brackets

Regular expressions include being able to match multiple letters contained in `[]`. These can be interpreted asking for a single character that matches at one element of the set inside the square brackets.

```
str_replace("banana", "[bn]", "f")
```

```
## [1] "fanana"
```

```
str_replace_all("banana", "[bn]", "f")
```

```
## [1] "fafafa"
```

```
str_replace_all("banana cream pie", "[an]", "e")
```

```
## [1] "beeeee creem pie"
```

We can get a range of letters using a hyphen. `[A-z]` represents all the capital letters and lower case letters.

```
str_replace_all("banana cream pie", "[a-z]", "e")
```

```
## [1] "eeeeee eeeee eee"
```

Notice that regular expressions are case sensitive

```
str_replace_all("banana Cream pie", "[A-Z]", "e")
```

```
## [1] "banana eream pie"
```

Negative search

Regular expressions default to finding a match. We can also do negative search, with things in `[^ ]`. Here we find all the letters that are not `i`.

```
str_replace_all("Mississippi", "[^i]", "s")
```

```
## [1] "sissssissi"
```

wildcards \\d \\s \\w

Regular expressions include wildcards for digits (`\\d`), spaces (`\\s`), and many characters (`\\w`) or any character (`.`)

```
str_replace("3 banana cream pies", "\\d", "4")
```

```
## [1] "4 banana cream pies"
```

```
str_replace("3 banana cream pies", "\\s", "-")
```

```
## [1] "3-banana cream pies"
```

```
str_replace("3 banana cream pies", "\\w", "9")
```

```
## [1] "9 banana cream pies"
```

```
str_replace("3 banana cream pies", ".", "2")
```

```
## [1] "2 banana cream pies"
```

```
str_replace("3 banana cream pies", "\\d\\s\\w", "B")
```

```
## [1] "Banana cream pies"
```

The opposite is done with capital letters

```
str_replace("3 banana cream pies", "\\D", "-")
```

```
## [1] "3-banana cream pies"
```

```
str_replace("3 banana cream pies", "\\S", "4")
```

```
## [1] "4 banana cream pies"
```

```
str_replace("3 banana cream pies", "\\W", "-")
```

```
## [1] "3-banana cream pies"
```

Repeated terms + * ?

Regular expressions become complicated with repeats.

+ matches 1 or more instances of a pattern that preceeds it.

```
str_replace("3 banana cream pies", "(an)+", "o")
```

```
## [1] "3 boa cream pies"
```

```
str_replace("3 bananana cream pies", "(an)+", "o")
```

```
## [1] "3 boa cream pies"
```

```
str_replace("3 bananana cream pies", "[A-z]+", "")
```

```
## [1] "3  cream pies"
```

* allows there to be 0 or more instances of a pattern that preceeds it.

```
str_replace("South Carolina", "South\\s*", "")
```

```
## [1] "Carolina"
```

```
str_replace("SouthCarolina", "South\\s*", "")
```

```
## [1] "Carolina"
```

```
str_replace("South-Carolina", "South\\s*", "")
```

```
## [1] "-Carolina"
```

Greedy vs Lazy evaluations

+ and * are 'greedy' meaning that they will search as far as they can:

```
str_extract("Alabama, Alaska, Arizona, Arkanasas, California,", "A.+,")
```

```
## [1] "Alabama, Alaska, Arizona, Arkanasas, California,"
```

If the command were lazy, it would stop after the first one it found. We can force these commands to stop at the first instance with a ? mark.

```
str_extract("Alabama, Alaska, Arizona, Arkanasas, California", "A.+?,")
```

```
## [1] "Alabama,"
```

For if you actually want to match on question marks or any other wildcard, you need to put \\ in front.

```
str_replace("Who? are you?", "\\?", "")
```

```
## [1] "Who are you?"
```

In practice you will find that using regular expressions will require going back to the help files to look up syntax, but they are a powerful technology in our research into text based data.

Exercise: Pickle Industry Recipe

Here we get the recipes from the pickle packer association:

```
PickleRecipesSite <- GET("http://web.archive.org/web/20170307183305/http://ilovepickles.org/book/export/
PickleRecipesText <- content(PickleRecipesSite, as = "text")
```

The recipes are one long character string.

We can use regular expressions to unpack these recipies.

First we will replace all the `\n`'s, These are new line commands, and not useful for us.

```
PickleRecipesText <- str_replace_all(PickleRecipesText, "[\n]", " ")
```

Ok, Suppose I want a list of all the ingredients: We can do that by looking for everything between What you need, and the end of the section, marked by `</div>`. I noticed a pattern reading it. Now we will use extract, which returns a nested list. We can undo that with an unlist command.

```
ingredients <- unlist(str_extract_all(PickleRecipesText, "What You Need: </div>.+?</div>"))

ingredients <- str_replace_all(ingredients, "<.+?>", ", ")
```

to seperate the ingredients, we "split" on commas, again, making a list.

```
ingredientslist <- str_split(ingredients, ",")
```

Remember, the map family of commands applies a function to each element of the list, and returns a list, Here we use `str_trim`.

```
trimingredients <- map(ingredientslist, str_trim)
```

Ok, now this next command is a little complicated. We calculate the list of each recipe with the map command, then we use that as the input into a rep command, which repeats each element of a vector a set number of times.

```
rep(c("a", "b"), c(1, 3))
```

```
## [1] "a" "b" "b" "b"
```

```
recipenumber <- rep(1:length(trimingredients), map(ingredientslist, length))
```

Now we pass it do a data frame

```
dfpickles <- data.frame(recipenumber = recipenumber, ingredient = unlist(trimingredients))
```

Cut out some stuff that is not interesting

```
dfpickles <- dfpickles %>% filter(!ingredient %in% c("What You Need: ", " ",
    "")) %>% mutate(ingredient = tolower(ingredient), ingredient = str_replace(ingredient,
    "[\\d/-]+\\s\\w+", ""))
```

```
## Warning: package 'bindrcpp' was built under R version 3.4.4
```

Problems

1) Use `group_by`, `str_detect`, and `ggplot` to display the prevelance of mayonaisse, saurkraut, and dill in the American Pickle Packer Association recipes.

2) Isaac Newton once claimed that among the following, **A** had the highest probability of success.

2a. Six fair dice are tossed independently and at least one '6' appears.

2b. Twelve fair dice are tossed independently and at least two '6's appear.

2c. Eighteen fair dice are tossed independently and at least three '6's appear.

Use R to evaluate Isaac Newton's claim (use simulation).

You can read more about this problem in Stigler, Stephen M. 2006 "Isaac Newton as a Probabilist" *Statistical Science* Vol. 21, No. 3 400-403.

3) Using Bayes Rule:

Suppose that we are studying a binomial phenomenon, such as the number of people in a precinct that vote in favor of Trump. Suppose θ is the probability that any one person votes for Trump.

The likelihood that we see a particular number of votes for Trump $y = 0, ..., n$ in the voting precinct is:

$$p(y|\theta) = \binom{n}{y}\theta^y(1-\theta)^{n-y}$$

Here n is the number of people eligible to vote in the precinct, which is fixed. This is the pdf of the data given the parameter θ, which can range from 0 to 1. We learned in class that we can use Bayes rule to reverse this conditional probability to discover $p(\theta|y)$, the pdf of θ.

$$p(\theta|y) = \frac{p(\theta)p(y|\theta)}{p(y)} = \frac{p(\theta)\binom{n}{y}\theta^y(1-\theta)^{n-y}}{\int_0^1 p(\theta)\binom{n}{y}\theta^y(1-\theta)^{n-y}d\theta}$$

What we need is $p(\theta)$, which we learned in class is our prior. Suppose we don't have strong beliefs about θ, we might want to use a prior that puts equal weight on all values of θ.

$$p(\theta) = \begin{cases} 1 \; for \; 0 \leq \theta \leq 1 \\ 0 \; else \end{cases}$$

This is called a uniform prior, it means we know nothing about θ. Plugging that into the prior equation we get the following:

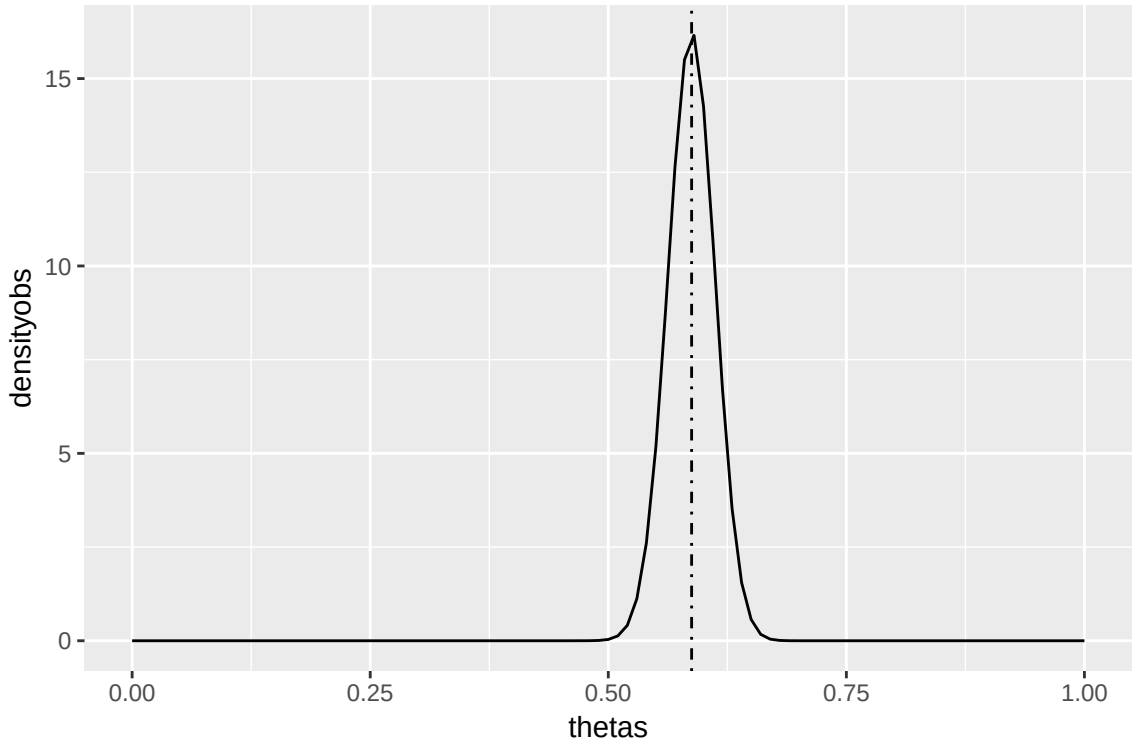
$$p(\theta|y) = \frac{\binom{n}{y}\theta^y(1-\theta)^{n-y}}{\int_0^1 \binom{n}{y}t^y(1-t)^{n-y}dt} = \frac{\theta^y(1-\theta)^{n-y}}{\int_0^1 t^y(1-t)^{n-y}dt}$$

We can write this in R as follows:

```
dposterior <- function(theta, y, n) {
    denominator <- function(t) {
        t^y * (1 - t)^(n - y)
    }

    integrated <- integrate(denominator, 0, 1)

    out <- (theta^y * (1 - theta)^(n - y))/integrated$value
    return(out)
}
```

Suppose we see that in our precinct of 400 eligible voters, we observe 235 votes for Trump: We get a fairly tight posterior distribution around (235/400).

```
simposterior <- data.frame(thetas = seq(0, 1, by = 0.01), densityobs = dposterior(seq(0,
    1, by = 0.01), 235, 400))

ggplot(simposterior, aes(thetas, densityobs)) + geom_freqpoly(stat = "identity") +
    geom_vline(xintercept = 235/400, linetype = "dotdash")
```



The formula for $p(\theta|y)$ is the Beta distribution:

$$Beta(a,b) = \frac{\theta^{a-1}(1-\theta)^{b-1}}{\int_0^1 t^{a-1}(1-t)^{b-1}dt}$$

Where $a = y + 1$ and $b = n - y + 1$.

```
dbeta(0.5, 235 + 1, 400 - 235 + 1)
```

```
## [1] 0.03441323
```

```
dposterior(0.5, 235, 400)
```

```
## [1] 0.03441213
```

3a. Kellyanne Conway is asked to report to the President about his support from the nation. She has available three samples - a focus group of 20 people, where 17 reported supporting Trump, a quick poll of 200 people, where 125 reported supporting Trump, and a big poll of 2000 people, where 1082 report supporting Trump. Use the Beta distribution to plot the posterior probability density function of an individual supporting Trump if we observe his support at 17 of 20, 125 of 200, or 1082 of 2000.

3b. Which of these should Kellyanne report to the President? Which is most heartening for the President? Which is 'fake news'?